Задание: создать базу данных и таблицы IMDb, загрузить тестовые данные из S3 и построить запросы a–f: жанры фильмов, фильмы без жанра, кросс-джойн, жанры без INNER JOIN, актёры фильма в заданном году и фильмы без жанра через ANTI JOIN.

Создание БД и таблиц

Листинг скрипта

```
-- Создание базы данных и таблиц IMDb
CREATE DATABASE IF NOT EXISTS imdb;
USE imdb;

DROP TABLE IF EXISTS imdb.actors;
DROP TABLE IF EXISTS imdb.genres;
DROP TABLE IF EXISTS imdb.movies;
DROP TABLE IF EXISTS imdb.roles;

CREATE TABLE imdb.actors
(
    id         UInt32,
    first_name String,
    last_name  String,
    gender     FixedString(1)
)
ENGINE = MergeTree
ORDER BY (id, first_name, last_name, gender);

CREATE TABLE imdb.genres
(
    movie_id UInt32,
    genre    String
)
ENGINE = MergeTree
ORDER BY (movie_id, genre);

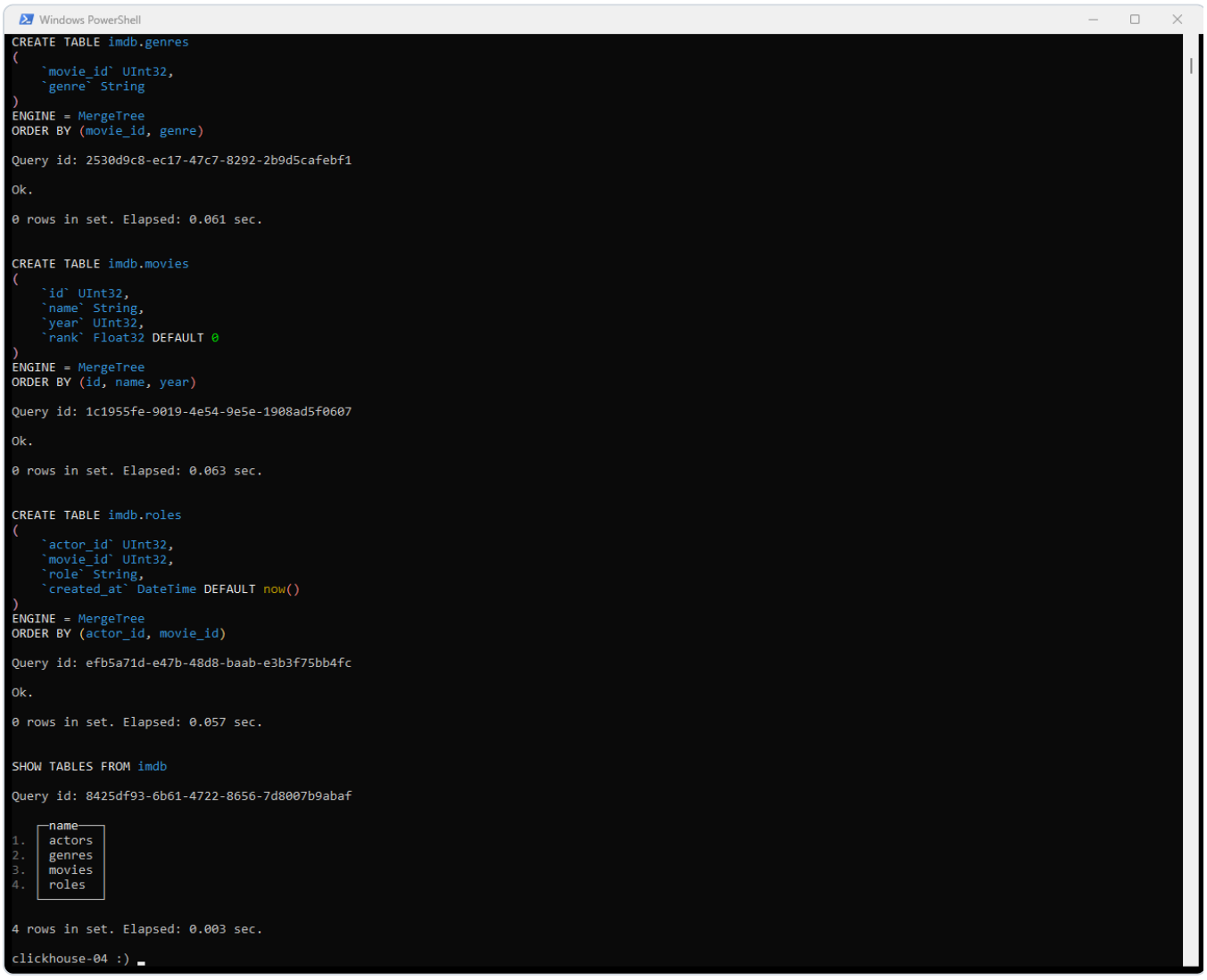
CREATE TABLE imdb.movies
(
    id   UInt32,
    name String,
    year UInt32,
    rank Float32 DEFAULT 0
)
ENGINE = MergeTree
ORDER BY (id, name, year);

CREATE TABLE imdb.roles
```

```
(
    actor_id   UInt32,
    movie_id   UInt32,
    role       String,
    created_at DateTime DEFAULT now()
)
ENGINE = MergeTree
ORDER BY (actor_id, movie_id);

SHOW TABLES FROM imdb;
```

Скриншот результата



Загрузка данных из S3

Листинг скрипта

```
-- Загрузка тестовых данных IMDb из S3
USE imdb;

INSERT INTO imdb.actors
SELECT *
FROM s3('https://datasets-documentation.s3.eu-west-3.amazonaws.com/imdb/imdb_ijs_actor
        'NOSIGN',
        'TSVWithNames');

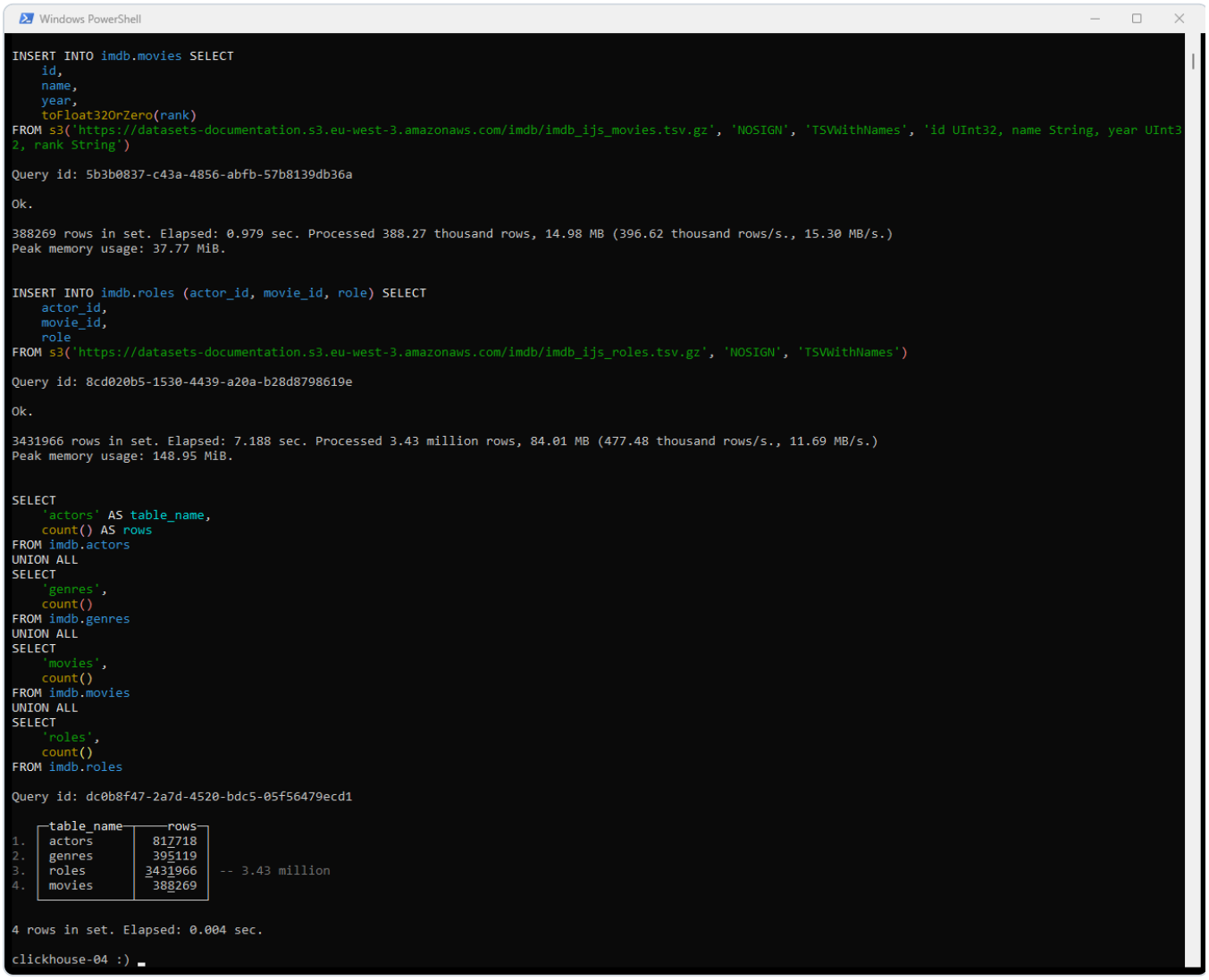
INSERT INTO imdb.genres
SELECT *
FROM s3('https://datasets-documentation.s3.eu-west-3.amazonaws.com/imdb/imdb_ijs_movie
        'NOSIGN',
        'TSVWithNames');

INSERT INTO imdb.movies
SELECT id, name, year, toFloat32OrZero(rank)
FROM s3('https://datasets-documentation.s3.eu-west-3.amazonaws.com/imdb/imdb_ijs_movie
        'NOSIGN',
        'TSVWithNames',
        'id UInt32, name String, year UInt32, rank String');

INSERT INTO imdb.roles(actor_id, movie_id, role)
SELECT actor_id, movie_id, role
FROM s3('https://datasets-documentation.s3.eu-west-3.amazonaws.com/imdb/imdb_ijs_roles
        'NOSIGN',
        'TSVWithNames');

SELECT 'actors' AS table_name, count() AS rows FROM imdb.actors
UNION ALL
SELECT 'genres', count() FROM imdb.genres
UNION ALL
SELECT 'movies', count() FROM imdb.movies
UNION ALL
SELECT 'roles', count() FROM imdb.roles;
```

Скриншот результата



Жанры для каждого фильма

Листинг скрипта

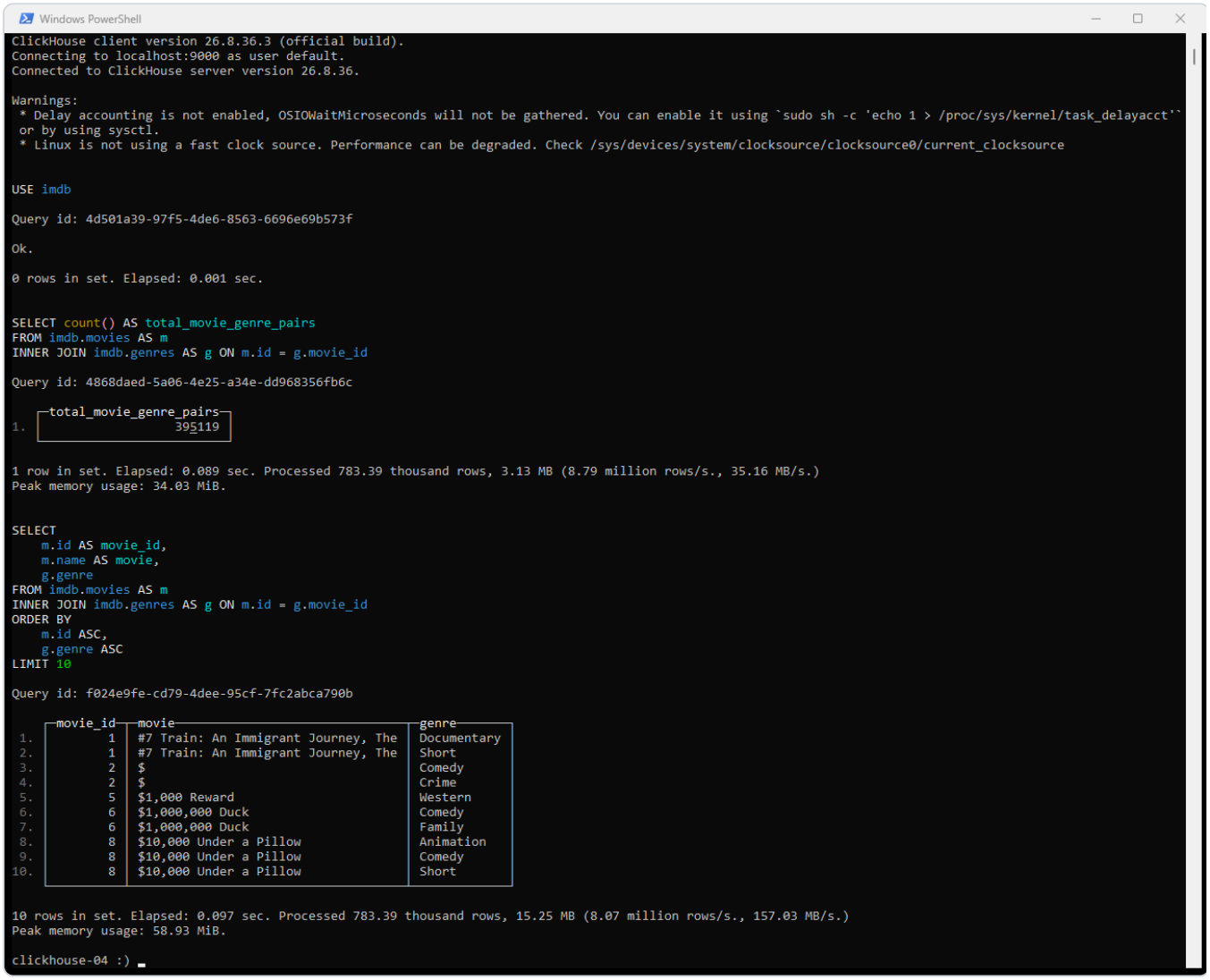
```sql
-- a. Жанры для каждого фильма
USE imdb;

SELECT count() AS total_movie_genre_pairs
FROM imdb.movies AS m
INNER JOIN imdb.genres AS g ON m.id = g.movie_id;

SELECT m.id AS movie_id, m.name AS movie, g.genre
FROM imdb.movies AS m
INNER JOIN imdb.genres AS g ON m.id = g.movie_id
```

```
ORDER BY m.id, g.genre
LIMIT 10;
```

Скриншот результата



Фильмы без жанра

Листинг скрипта

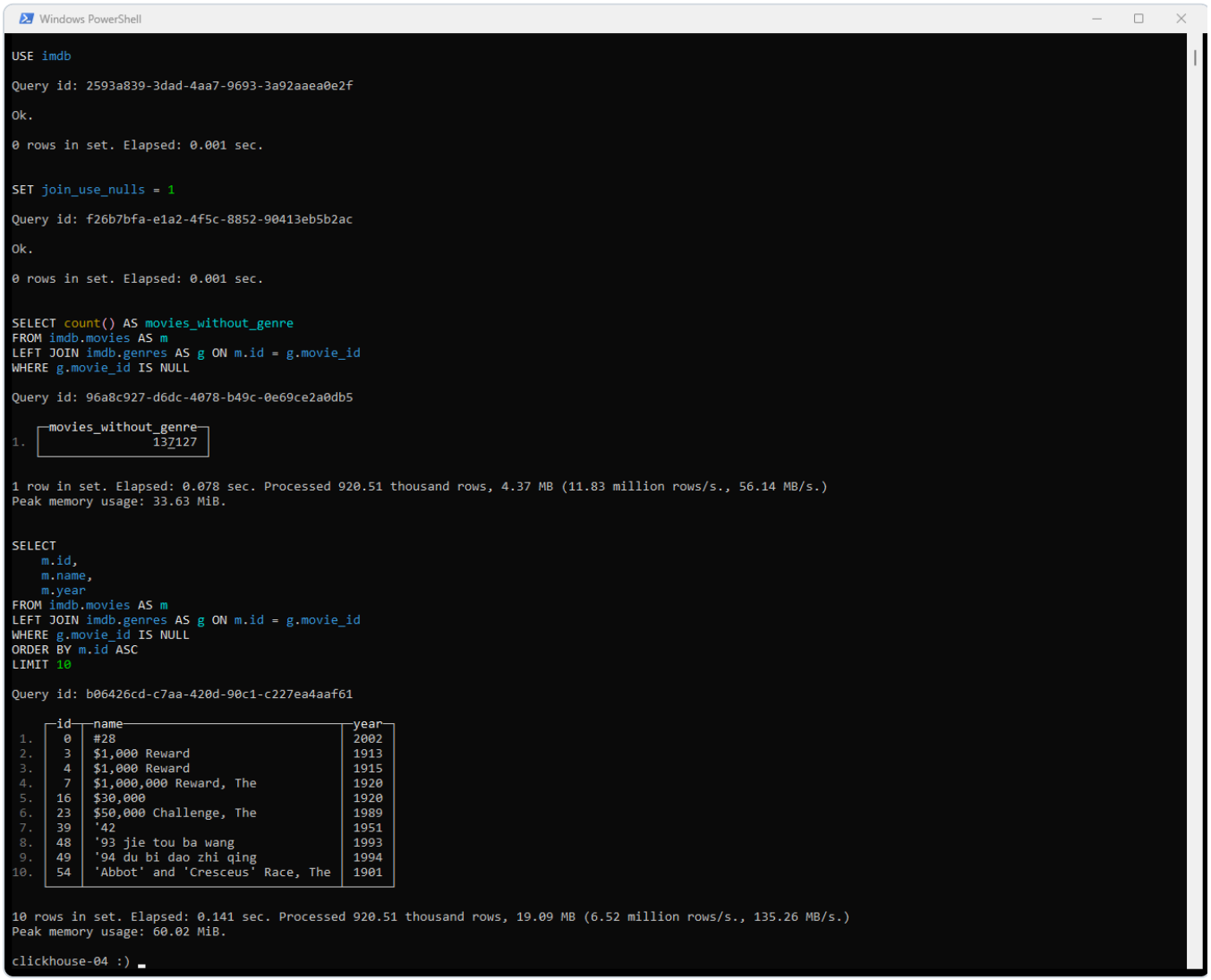
```
-- b. Все фильмы, у которых нет жанра
USE imdb;
SET join_use_nulls = 1;

SELECT count() AS movies_without_genre
FROM imdb.movies AS m
LEFT JOIN imdb.genres AS g ON m.id = g.movie_id
```

```
WHERE g.movie_id IS NULL;

SELECT m.id, m.name, m.year
FROM imdb.movies AS m
LEFT JOIN imdb.genres AS g ON m.id = g.movie_id
WHERE g.movie_id IS NULL
ORDER BY m.id
LIMIT 10;
```

Скриншот результата



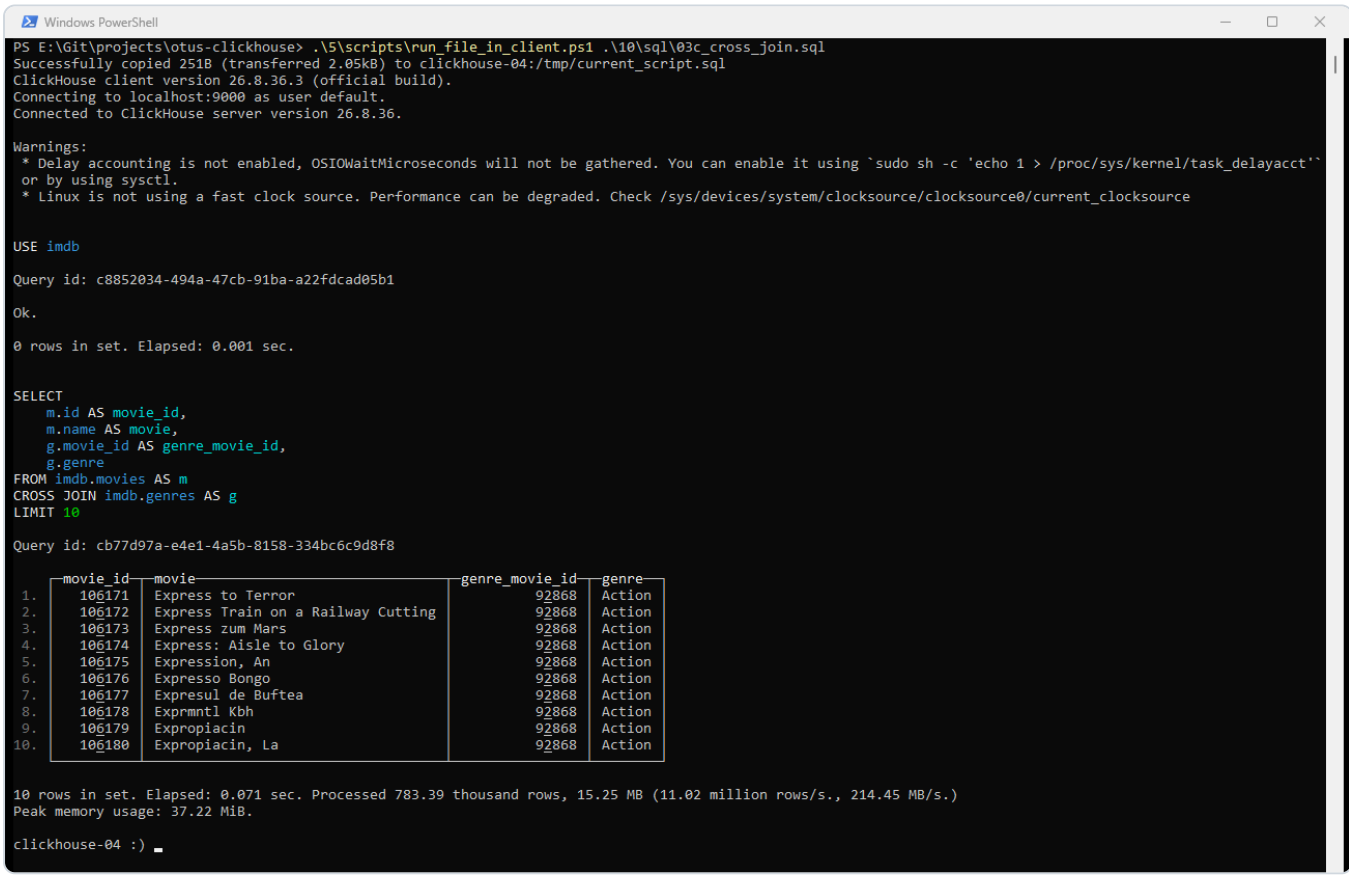
Объединение фильмов и жанров

Листинг скрипта

```sql
-- c. Объединение каждой строки movies с каждой строкой genres
USE imdb;

SELECT m.id AS movie_id, m.name AS movie, g.movie_id AS genre_movie_id, g.genre
FROM imdb.movies AS m
CROSS JOIN imdb.genres AS g
LIMIT 10;
```

Скриншот результата



```
Windows PowerShell
PS E:\Git\projects\otus-clickhouse> .\5\scripts\run_file_in_client.ps1 .\10\sql\03c_cross_join.sql
Successfully copied 251B (transferred 2.05kB) to clickhouse-04:/tmp/current_script.sql
ClickHouse client version 26.8.36.3 (official build).
Connecting to localhost:9000 as user default.
Connected to ClickHouse server version 26.8.36.

Warnings:
 * Delay accounting is not enabled, OSIOWaitMicroseconds will not be gathered. You can enable it using `sudo sh -c 'echo 1 > /proc/sys/kernel/task_delayacct'`
 or by using sysctl.
 * Linux is not using a fast clock source. Performance can be degraded. Check /sys/devices/system/clocksource/clocksource0/current_clocksource


USE imdb

Query id: c8852034-494a-47cb-91ba-a22fdcad05b1

Ok.

0 rows in set. Elapsed: 0.001 sec.


SELECT
    m.id AS movie_id,
    m.name AS movie,
    g.movie_id AS genre_movie_id,
    g.genre
FROM imdb.movies AS m
CROSS JOIN imdb.genres AS g
LIMIT 10

Query id: cb77d97a-e4e1-4a5b-8158-334bc6c9d8f8

    ┌─movie_id─┬─movie──────────────────────────┬─genre_movie_id─┬─genre──┐
 1. │   106171 │ Express to Terror              │          92868 │ Action │
 2. │   106172 │ Express Train on a Railway Cutting │          92868 │ Action │
 3. │   106173 │ Express zum Mars               │          92868 │ Action │
 4. │   106174 │ Express: Aisle to Glory        │          92868 │ Action │
 5. │   106175 │ Expression, An                 │          92868 │ Action │
 6. │   106176 │ Expresso Bongo                 │          92868 │ Action │
 7. │   106177 │ Expresul de Buftea             │          92868 │ Action │
 8. │   106178 │ Exprmntl Kbh                   │          92868 │ Action │
 9. │   106179 │ Expropiacin                    │          92868 │ Action │
10. │   106180 │ Expropiacin, La                │          92868 │ Action │
    └──────────┴────────────────────────────────┴────────────────┴────────┘

10 rows in set. Elapsed: 0.071 sec. Processed 783.39 thousand rows, 15.25 MB (11.02 million rows/s., 214.45 MB/s.)
Peak memory usage: 37.22 MiB.

clickhouse-04 :)
```

Жанры для каждого фильма без INNER JOIN

Листинг скрипта

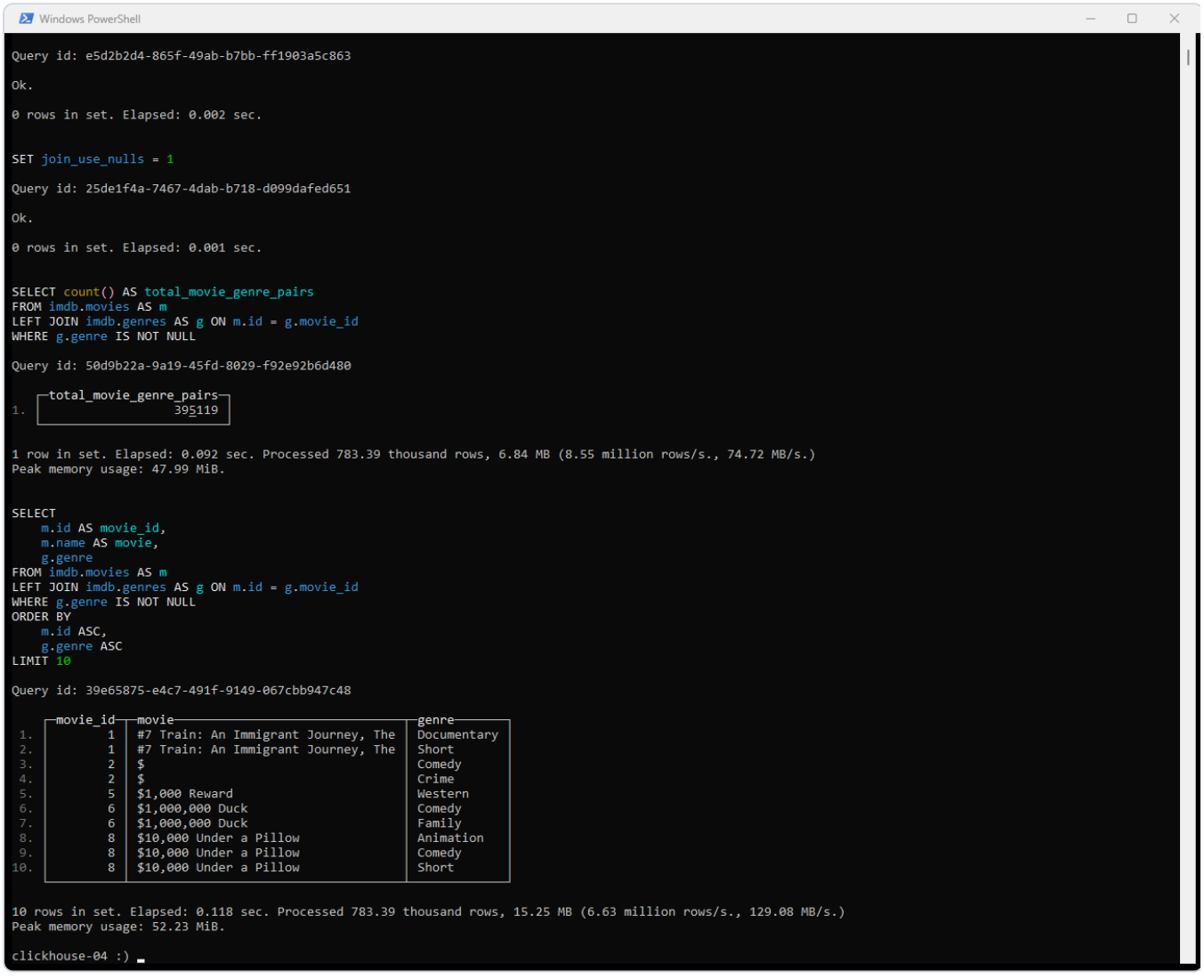
```sql
-- d. Жанры для каждого фильма без INNER JOIN
USE imdb;
```

```sql
SET join_use_nulls = 1;

SELECT count() AS total_movie_genre_pairs
FROM imdb.movies AS m
LEFT JOIN imdb.genres AS g ON m.id = g.movie_id
WHERE g.genre IS NOT NULL;

SELECT m.id AS movie_id, m.name AS movie, g.genre
FROM imdb.movies AS m
LEFT JOIN imdb.genres AS g ON m.id = g.movie_id
WHERE g.genre IS NOT NULL
ORDER BY m.id, g.genre
LIMIT 10;
```

Скриншот результата



Актеры и актрисы в фильмах 2000 года

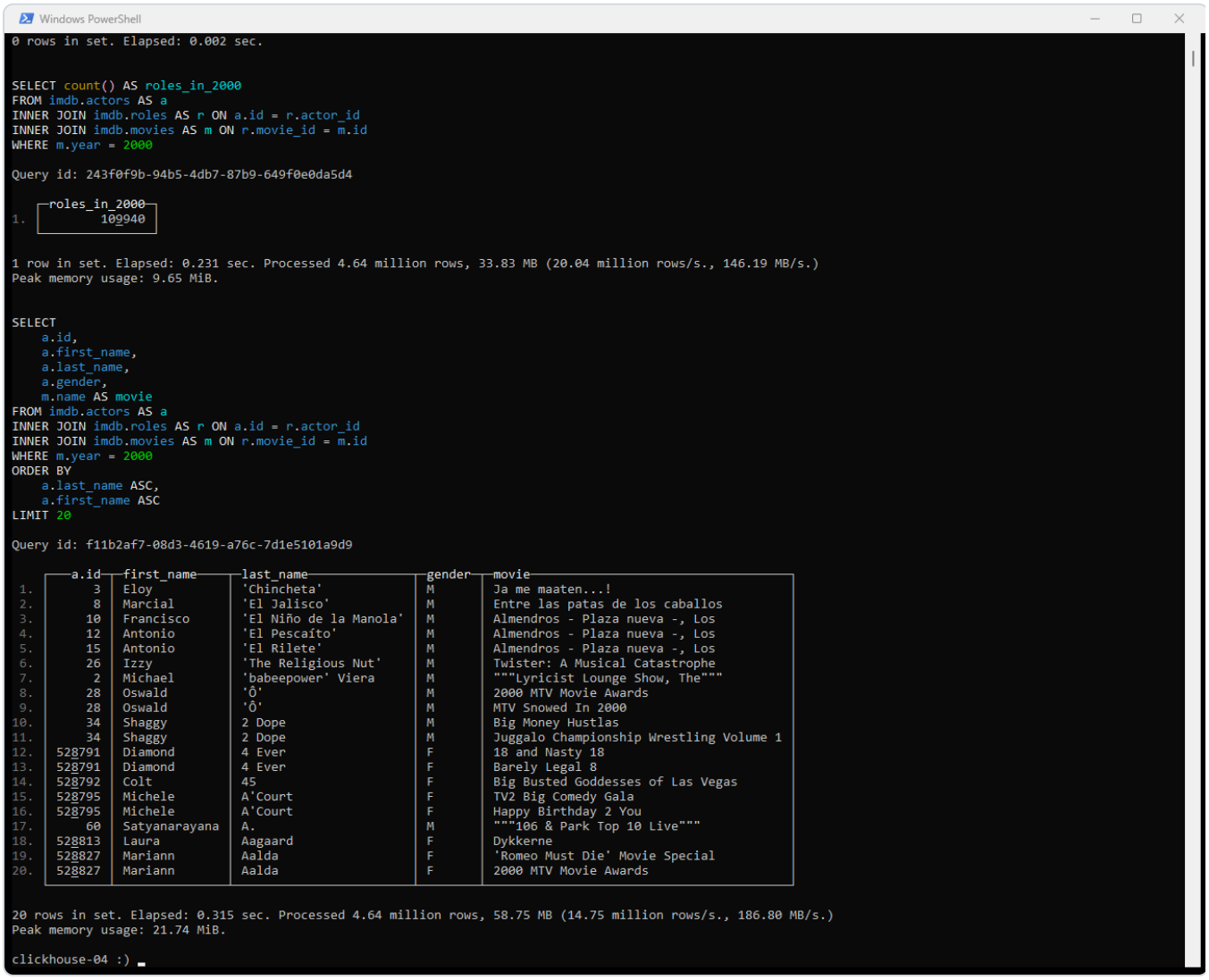
Листинг скрипта

```
-- e. Актеры и актрисы, снявшиеся в фильме в 2000 году
USE imdb;

SELECT count() AS roles_in_2000
FROM imdb.actors AS a
INNER JOIN imdb.roles AS r ON a.id = r.actor_id
INNER JOIN imdb.movies AS m ON r.movie_id = m.id
WHERE m.year = 2000;

SELECT a.id, a.first_name, a.last_name, a.gender, m.name AS movie
FROM imdb.actors AS a
INNER JOIN imdb.roles AS r ON a.id = r.actor_id
INNER JOIN imdb.movies AS m ON r.movie_id = m.id
WHERE m.year = 2000
ORDER BY a.last_name, a.first_name
LIMIT 20;
```

Скриншот результата



Фильмы без жанра через ANTI JOIN

Листинг скрипта

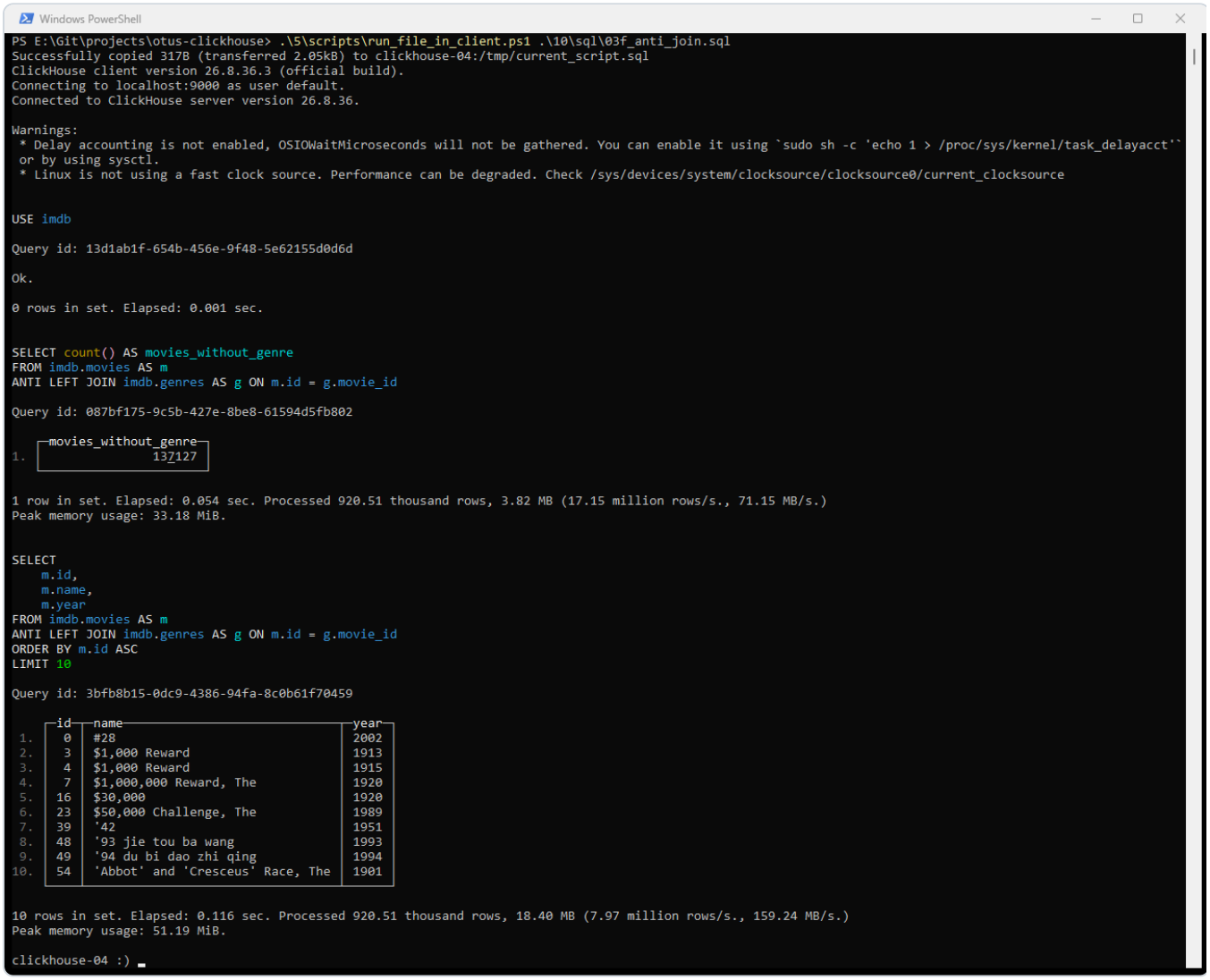
```sql
-- f. Фильмы без жанра через LEFT ANTI JOIN
USE imdb;

SELECT count() AS movies_without_genre
FROM imdb.movies AS m
LEFT ANTI JOIN imdb.genres AS g ON m.id = g.movie_id;

SELECT m.id, m.name, m.year
FROM imdb.movies AS m
LEFT ANTI JOIN imdb.genres AS g ON m.id = g.movie_id
```

```
ORDER BY m.id
LIMIT 10;
```

Скриншот результата



Проблемы и решения

1. **Доступ к S3.** Исходный вызов `s3(url, 'TSVWithNames')` использует серверные учётные данные и завершается ошибкой `ACCESS_DENIED`. Для публичного бакета добавлен режим `NOSIGN`:

```sql
s3(url, 'NOSIGN', 'TSVWithNames')
```

2. **Пустые значения `rank` в movies.** При автоматическом определении схемы `TSVWithNames` колонка `rank` получает тип `Float32`, и пустые значения вызывают `CANNOT_PARSE_NUMBER`. Решение — явно задать структуру с `rank String` и преобразовать значение:

```sql
SELECT id, name, year, toFloat32OrZero(rank)
FROM s3(url, 'NOSIGN', 'TSVWithNames', 'id UInt32, name String, year UInt32, rank String');
```

3. **`LEFT JOIN` и `NULL`.** По умолчанию в ClickHouse `join_use_nulls = 0`, поэтому у несопоставленных строк правой таблицы значения не равны `NULL`, а равны значениям по умолчанию. Из-за этого запрос b возвращал `0`, а запрос d — `532246` вместо `395119`. Решение — включить `SET join_use_nulls = 1;` перед `LEFT JOIN`. После этого b возвращает `137127` (совпадает с ANTI JOIN в f), а d — `395119` (совпадает с INNER JOIN в a).

4. **Кросс-джойн.** `movies` × `genres` даёт около `1.53e11` строк, поэтому полный `count()` не выполнялся; для отчёта использован `LIMIT 10`.

5. **Год в запросе e.** В запросе e зафиксирован 2000 год; при необходимости год заменяется в условии `WHERE m.year = 2000`. Всего за 2000 год найдено `10940` ролей.